COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	8/4/2026	**DATE SUBMITTED:**	8/7/2026
NAME:	Estorco, Clint Hoven A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 Hands-on Activity 4.1 STACKS

1. PROCEDURE OUTPUT

ILO_A

INPUT

```cpp
#include <iostream>
#include <queue>


int main (){
  //create an object
  std::queue<char> myQ;

void display(std::queue<char> copyQ);

  //use the queue operation
  myQ.push('C');
  myQ.push('L');
  myQ.push('I');
  myQ.push('N');
  myQ.push('T');
  std::cout<<"My current front is: " << myQ.front()<<std::endl;
  std::cout<<"My current back is: " << myQ.back()<<std::endl;



  //use the dequeue opearation
  myQ.pop();
  display(myQ);
  myQ.pop();
  display(myQ);
  myQ.pop();




  return 0;


}

  void display(std::queue<char> copyQ){

    //create a copy of a queue
    std::queue<char> temp = copyQ;

    //loop until empty
    while(!temp.empty()){
```

OUTPUT

```
PS C:\Users\TIPQC\Desktop\Estorco_Queue-HOA5p1\ILOA> C:\
My current front is: C
My current back is: T
 L  I  N  T
  I  N  T
PS C:\Users\TIPQC\Desktop\Estorco_Queue-HOA5p1\ILOA>
```

```
        while(!temp.empty()){
          //Display Front
          std::cout<<" "<< temp.front() << " ";

          //dequeu the front
          temp.pop();



        }

        //add a new line
        std::cout<<std::endl;


    }
```

ILO_B	
INPUT	OUTPUT

```
#include <iostream>
#include "queueLL.h"


int main (){
  qNode<char>* front = nullptr;
  qNode<char>* back = nullptr;

  enqueue('C', &front, &back);
  std::cout<<front ->data << " "<<"Back: "<<back->data<<std::endl;
  enqueue('L', &front, &back);
  std::cout<<front ->data << " "<<"Back: "<<back->data<<std::endl;
  enqueue('I', &front, &back);
  std::cout<<front ->data << " "<<"Back: "<<back->data<<std::endl;
  enqueue('N', &front, &back);
  std::cout<<front ->data << " "<<"Back: "<<back->data<<std::endl;
  enqueue('T', &front, &back);
  std::cout<<front ->data << " "<<"Back: "<<back->data<<std::endl;

  std::cout<<"testing the dequeue operator: \n";
 dequeue(&front, &back);
  std::cout<<"Front: " << front ->data << " "<<"Back: "<<back->data<<std::endl;
  dequeue(&front, &back);
  std::cout<<"Front: " << front ->data << " "<<"Back: "<<back->data<<std::endl;
  dequeue(&front, &back);
  std::cout<<"Front: " << front ->data << " "<<"Back: "<<back->data<<std::endl;
  dequeue(&front, &back);
  std::cout<<"Front: " << front ->data << " "<<"Back: "<<back->data<<std::endl;
  dequeue(&front, &back);
  std::cout<<"Front: " << front ->data << " "<<"Back: "<<back->data<<std::endl;
  dequeue(&front, &back);
  dequeue(&front, &back);



  return 0;
}
```

queueLL.h

```
C:\msys64\ucrt64\bin>C:\Users\analy\Downloads\Estorco_Queue-HOA5p1\ILOB\main.exe
C Back: C
C Back: L
C Back: I
C Back: N
C Back: T

Displaying full queue:
Queue: C L I N T
```

```
testing the dequeue operator:
Front: L Back: T
Queue: L I N T
Front: I Back: T
Queue: I N T
Front: N Back: T
Queue: N T
Front: T Back: T
Queue: T
the queue is empty
```

main.cpp ILOB | main.cpp ILOA | queueLL.h

ILOB > queueLL.h > ...

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

//CREATE A CLASS NODE
template <typename T>
class qNode{
  public:
  T data;
  qNode* next;
};


//NEW NODE CREATOR
template <typename T>
qNode<T>* new_node(T newData){
  //allocate a space for new data
  qNode<T>* newNode = new qNode<T>;
  //store new data to the newnode
  newNode -> data  = newData;

  //point the newnode to the next
  newNode -> next = nullptr;

  return newNode;
}

//ENQUEUE
template<typename T>
void enqueue(T newData, qNode<T>** frontptr, qNode<T>** backptr) {
```

```cpp
void enqueue(T newData, qNode<T>** frontptr, qNode<T>** backptr) {

  //CREATE A NEW NODE
  qNode<T>* newNode = new_node(newData);

  //check if the queue is empty
  if((*frontptr)== nullptr && (*backptr) == nullptr){
    (*frontptr) = newNode;
    (*backptr) = newNode;
    return;
  }
  //if not then insert at the bACK
  (*backptr) ->next = newNode;
  (*backptr) = newNode;



}
//DEQUEUE
template <typename T>
void dequeue(qNode<T>** frontptr, qNode<T>** backptr){
  //check if the queue is empty
  if((*frontptr) == nullptr && (*backptr) == nullptr){
    std::cout<<"the queue is empty"<<std::endl;
    return;
  }
```

```cpp
//create a temporary variable to tore the node to the deleted
qNode<T>* deleteTemp = nullptr;

//Assign the current front to the deleteTemp
deleteTemp = (*frontptr);

//check if the queue has only 1 item
if((*frontptr)-> next == nullptr && (*backptr) -> next == nullptr){
  //means only 1 item is insidie the queue
  (*frontptr) = nullptr;
  (*backptr) = nullptr;
  delete  deleteTemp;
  return;
}

(*frontptr) = (*frontptr)->next;
deleteTemp->next = nullptr;

//delete na node
delete deleteTemp;
}


//DISPLAY ALL
template <typename T>
void display(qNode<T>** frontptr) {
  //check if the queue is empty
```

```
83    //check if the queue is empty
84    if ((*frontptr) == nullptr) {
85      std::cout << "the queue is empty" << std::endl;
86      return;
87    }
88
89    //create a temporary pointer to traverse the queue from front to ba
90    qNode<T>* temp = (*frontptr);
91
92    std::cout << "Queue: ";
93    while (temp != nullptr) {
94      std::cout << temp->data << " ";
95      temp = temp->next;
96    }
97    std::cout << std::endl;
98  }
99
100 #endif
```

main .cpp/QUEUE.h	
INPUT	OUTPUT

MAIN CPP

```
1   #include <iostream>
2   #include "QueueArr.h"
3
4   int main() {
5       // creating an object
6       QueueArr<int> q(5);
7
8       // enqueue 10, 20, 30, 40, 50:
9       q.Enqueue(1);
10
11      std::cout << "Front: " << q.Front() << std::endl;
12      std::cout << "Back: " << q.Back() << std::endl;
13      std::cout << "Size: " << q.Size() << std::endl;
14
15      q.Enqueue(2);
16
17      std::cout << "Front: " << q.Front() << std::endl;
18      std::cout << "Back: " << q.Back() << std::endl;
19      std::cout << "Size: " << q.Size() << std::endl;
20
21      q.Enqueue(3);
22      q.Enqueue(4);
23
24      std::cout << "Front: " << q.Front() << std::endl;
25      std::cout << "Back: " << q.Back() << std::endl;
26      std::cout << "Size: " << q.Size() << std::endl;
27
28      q.Enqueue(5);
29
30      std::cout << "Front: " << q.Front() << std::endl;
31      std::cout << "Back: " << q.Back() << std::endl;
```

```
32      std::cout << "Size: " << q.Size() << std::endl;
33
34      q.Enqueue(6); // Will display "Queue is full"
35
36      std::cout << "\nremoved: " << q.Dequeue() << std::endl;
37      std::cout << "removed: " << q.Dequeue() << std::endl;
38
39      q.clear();
40
41      std::cout << "is the queue empty? " << (q.Empty() ? "yes" : "no") << std::endl;
42
43      return 0;
44  }
```

Queue.h

```
C:\msys64\ucrt64\bin>C:\Users\analy\OneDrive\
Front: 1
Back: 1
Size: 1
Front: 1
Back: 2
Size: 2
Front: 1
Back: 4
Size: 4
Front: 1
Back: 5
Size: 5
```

```
Front: 1
Back: 4
Size: 4
Front: 1
Back: 5
Size: 5
Queue is full

removed: 1
removed: 2
is the queue empty? yes

C:\msys64\ucrt64\bin>
```

QueueArr.h > operator=(const QueueArr<T>&)

```h
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <iostream>

//ARRAY BASED QUEUE IMPLEMENTATION
template <typename T>
class QueueArr{
    private:
    //dynamic array allocate the queue
    T* q_array;

    //maximum number of elements in the queue
    size_t q_capacity;
    //current number of elements in the queue
    size_t q_size;
    //index of the front element in the queue
    int q_front;
    //index of the back element in the queue
    int q_back;


    public:
    //constructor
    QueueArr(size_t capacity = 10);
    //copy constructor
    QueueArr(const QueueArr<T>& other);
    //copy assignment operator
    QueueArr<T>& operator=(const QueueArr<T>& other);
    //destructor
    ~QueueArr();
    //queue operator
    bool Empty();
    bool Full();

    size_t Size();

    void clear();
    T Front();
    T Back();
    void Enqueue(T value);
    T Dequeue();
};

// Constructor
template <typename T>
QueueArr<T>::QueueArr(size_t capacity) {
    q_capacity = capacity;
    q_array = new T[q_capacity];
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

// Copy Constructor
template <typename T>
QueueArr<T>::QueueArr(const QueueArr<T>& other) {
    q_capacity = other.q_capacity;
    q_size = other.q_size;
```

```
    q_front = other.q_front;
    q_back = other.q_back;
    q_array = new T[q_capacity];
    for (size_t i = 0; i < q_capacity; i++) {
        q_array[i] = other.q_array[i];
    }
}

// Copy Assignment Operator
template <typename T>
QueueArr<T>& QueueArr<T>::operator=(const QueueArr<T>& other) {
    if (this != &other) {
        delete[] q_array;
        q_capacity = other.q_capacity;
        q_size = other.q_size;
        q_front = other.q_front;
        q_back = other.q_back;
        q_array = new T[q_capacity];
        for (size_t i = 0; i < q_capacity; i++) {
            q_array[i] = other.q_array[i];
        }
    }
    return *this;
}

// Destructor
template <typename T>
QueueArr<T>::~QueueArr() {
    delete[] q_array;
}
```

```
}

//empty
template <typename T>
bool QueueArr<T>::Empty(){
    //check if the size if zero
    return q_size == 0;
}

//full
template <typename T>
bool QueueArr<T>::Full(){
    //check size if its max capacity
    return q_size == q_capacity;
}

//size_t
template <typename T>
size_t QueueArr<T>::Size(){
    return q_size;
}

// Front
template <typename T>
T QueueArr<T>::Front(){
    if(Empty()){
        std::cout<<"Queue is Empty\n";
        return T();
    }
```

```
    return q_array[q_front];
}

// Back
template<typename T>
T QueueArr<T>::Back(){
    //Check if the queue is empty
    if(Empty()){
        std::cout<<"Queue is empty"<<std::endl;
        return T();
    }
    //return to the back
    return q_array[q_back];
}

//clear all
template <typename T>
void QueueArr<T>::clear(){
    //reset the size and front and back index
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

//Void Enqueue
template <typename T>
void QueueArr<T>::Enqueue(T value){
```

```
144 void QueueArr<T>::Enqueue(T value){
145
146     //check if the queue is full
147     if(Full()){
148         std::cout<<"Queue is full\n";
149         return;
150     }
151     //move q_back circularly
152     q_back = (q_back + 1) % q_capacity;
153
154     //store  the value to the back
155     q_array[q_back] = value;
156
157     //increment the q_size
158     q_size++;
159 }
160
161 //T Dequeue
162 template <typename T>
163 T QueueArr<T>::Dequeue(){
164     //check if the queue is empty
165     if(Empty()){
166         std::cout<<"Queue is empty\n";
167         return T();
168     }
169     //store the front value to return
170     T value = q_array[q_front];
171
172     //move q_front circularly
173     q_front = (q_front + 1) % q_capacity;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
172     //move q_front circularly
173     q_front = (q_front + 1) % q_capacity;
174
175     //decrement the size
176     q_size--;
177
178     // reset indexes if queue becomes empty
179     if(Empty()) clear();
180
181     //return to temporary value
182     return value;
183 }
184
185 #endif
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

INPUT	OUTPUT

COMPUTER ENGINEERING

TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

QUEZON CITY

PrintJobQueue.cpp > Printer > processJobs()

```cpp
#include <iostream>
#include <string>

//Create a class
class Job {
public:
    int jobID;
    std::string userName;
    int numPages;

    // Constructors
    Job() : jobID(0), userName(""), numPages(0) {}
    Job(int id, std::string name, int pages) : jobID(id), userName(name), numPages(pages) {}
};
// A Node class to hold the Job data and the pointer for our linked list queue
class Node {
public:
    Job data;
    Node* next;

    Node(Job j) : data(j), next(nullptr) {}
};
//Create another class called Printer
class Printer {
private:
    Node* front; // Points to the front of the queue (next to be printed)
    Node* rear;  // Points to the back of the queue (newly added jobs)

public:
    Printer() : front(nullptr), rear(nullptr) {}
    // Method to add new jobs to the queue
```

```cpp
    // Method to add new jobs to the queue
    void addJob(int id, std::string name, int pages) {
        Job newJob(id, name, pages);
        Node* newNode = new Node(newJob);

        // If the queue is empty, the new job is both the front and the rear
        if (rear == nullptr) {
            front = rear = newNode;
        } else {
            // Otherwise, add to the back and update the rear pointer
            rear->next = newNode;
            rear = newNode;
        }
        std::cout << "System: Added Job ID " << id << " submitted by " << name << " (" << pages << " pages) to the queue.\n";
    }
    // Process all jobs using a first come, first served strategy
    void processJobs() {
        if (front == nullptr) {
            std::cout << "The printer queue is currently empty.\n";
            return;
        }
        std::cout << "\n--- Waking up printer... Processing queue ---\n";

        // Loop until all jobs are processed
        while (front != nullptr) {
            Node* temp = front;
            // Simulate processing the job
            std::cout << "[PRINTING] Job ID: " << temp->data.jobID
```

```cpp
                      << " | User: " << temp->data.userName
                      << " | Pages: " << temp->data.numPages << "...\n";

            // Move the front pointer to the next job in line
            front = front->next;
            // If the queue becomes empty after dequeueing, reset rear to nullptr
            if (front == nullptr) {
                rear = nullptr;
            }
            // Free the memory
            delete temp;
        }
        std::cout << "All pending jobs have been successfully processed.";
    }
};
int main() {
    Printer officePrinter;
    // Step 4: Simulate a scenario where multiple people are adding jobs
    std::cout << "Employees sending print requests \n";
    officePrinter.addJob(1001, "Clint", 5);
    officePrinter.addJob(1002, "John", 12);
    officePrinter.addJob(1003, "Evelyn", 1);
    officePrinter.addJob(1004, "Lucia", 45);

    // The printer processes them one by one
    officePrinter.processJobs();
```

```cpp
    // The printer processes them one by one
    officePrinter.processJobs();

    return 0;
}
```

```
Employees sending print requests
System: Added Job ID 1001 submitted by Clint (5 pages) to the queue.
System: Added Job ID 1002 submitted by John (12 pages) to the queue.
System: Added Job ID 1003 submitted by Evelyn (1 pages) to the queue.
System: Added Job ID 1004 submitted by Lucia (45 pages) to the queue.

--- Waking up printer... Processing queue ---
[PRINTING] Job ID: 1001 | User: Clint | Pages: 5...
[PRINTING] Job ID: 1002 | User: John | Pages: 12...
[PRINTING] Job ID: 1003 | User: Evelyn | Pages: 1...
[PRINTING] Job ID: 1004 | User: Lucia | Pages: 45...
All pending jobs have been successfully processed.
C:\msys64\ucrt64\bin>
```

ANALYSIS:

This output shows the printing jobs printed in the order they were entered (Clint, John, Evelyn, Lucia) with a First-In, First-Out (FIFO) queueing mechanism. This is because the `addJob` is constantly adding jobs to the end of the linked list and the `processJobs` is constantly removing jobs from the front.

QUESTION ANALYSIS

I chose the linked list because its size can be adjusted as the size of the printed material changes. It also can be efficiently dequeued by updating the pointers without having to worry about shifting array elements after each job as in the case of using an array.

CONCLUSION:

This activity was a successful demonstration of the fundamental principles and operation of the Queue data structure using the C++ Standard Template Library (STL) Queue data structure, as well as a custom implementation of the Queue data structure with Linked List data structure. The shared printer simulation was used to practically test the First-In, First-Out (FIFO) scheduling strategy, which verified that the tasks are processed in the same order as they were added to a queue. In conclusion, the dynamic linked list representation was the better way to represent the job list compared to the fixed array representation because it did not waste memory on arbitrary job list sizes and did not require shuffling array elements to dequeue a job with $O(1)$ time complexity.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.